SIMATS ENGINEERING

Assessment Tool 1 — Class Test: Model Answers

Course Code: CSA14 | Course Name: Compiler Design

CO4: Examine intermediate code generation techniques to enhance code optimization (BL4)

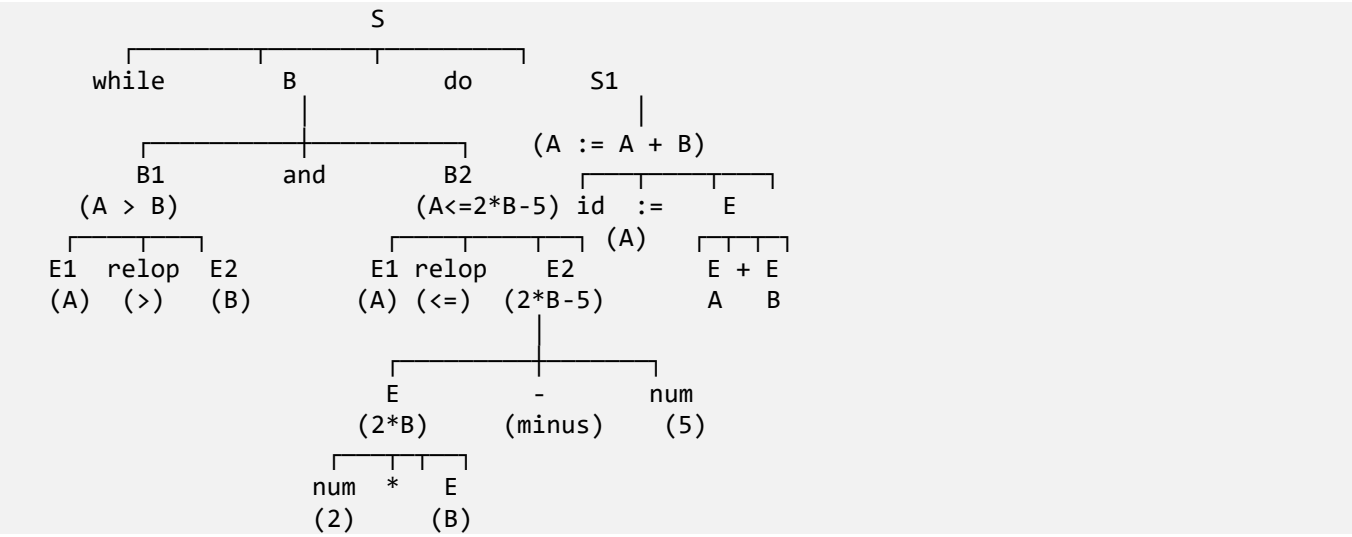
Q1. While-statement: while A > B and A <= 2*B - 5 do A := A + B

(i) Parse Tree

Using the grammar:

```
S  → while ( B ) do S1
B  → B1 and B2   |   E1 relop E2
E  → E + E  |  E * E  |  id  |  num
```

Parse tree for the statement:



(ii) Intermediate Code (Three Address Code) for the while-statement

Using jumping/short-circuit code for the Boolean expression:

```
100:  if A > B goto 102
101:  goto 108              // exit loop (B is false)
102:  t1 = 2 * B
103:  t2 = t1 - 5
104:  if A <= t2 goto 106
105:  goto 108              // exit loop (B is false)
106:  t3 = A + B
107:  A  = t3
       goto 100             // re-test loop condition
108:  ...                   // next statement after loop
```

Q2. Translation Scheme for the Assignment Statement

Grammar and semantic actions (S-attributed translation scheme):

```
S → id := E      { p = lookup(id.name);
                   if p ≠ nil then emit(p '=' E.place)
                   else error('undeclared variable') }

E → E1 + E2      { E.place = newtemp();
                   emit(E.place '=' E1.place '+' E2.place) }

E → E1 * E2      { E.place = newtemp();
                   emit(E.place '=' E1.place '*' E2.place) }

E → - E1         { E.place = newtemp();
                   emit(E.place '=' 'uminus' E1.place) }

E → ( E1 )       { E.place = E1.place }

E → id           { E.place = id.place }
```

Three Address Code for x = (a + b) * (c + d):

```
t1 = a + b
t2 = c + d
t3 = t1 * t2
x  = t3
```

Q3. Arithmetic Expression: a + b*c/e^f + b*a

(i) Operator precedence and associativity

- ^ (exponentiation) — highest precedence, right-associative
- * and / — next precedence, left-associative
- + — lowest precedence, left-associative

Evaluation order dictated by these rules: e^f is evaluated first (highest precedence). Then b*c is computed and divided by the result of e^f (left-to-right for * and /). This value is added to a. Separately, b*a is evaluated, and finally the two partial sums are added left-to-right, since + is left-associative.

(ii) Three Address Code (using temporaries)

```
t1 = e ^ f
t2 = b * c
t3 = t2 / t1
t4 = a + t3
t5 = b * a
t6 = t4 + t5      // t6 holds the final result
```

(iii)(a) Quadruples (op, arg1, arg2, result)

#	op	arg1	arg2	result
0	^	e	f	t1
1	*	b	c	t2
2	/	t2	t1	t3
3	+	a	t3	t4
4	*	b	a	t5
5	+	t4	t5	t6

(iii)(b) Triples (index, op, arg1, arg2)

#	op	arg1	arg2
(0)	^	e	f
(1)	*	b	c
(2)	/	(1)	(0)
(3)	+	a	(2)
(4)	*	b	a
(5)	+	(3)	(4)

(iii)(c) Indirect Triples (pointer-based)

Statement list (order of execution — pointers into the triple table):

```
Stmt#   Pointer
(14)     →  (100)
(15)     →  (101)
(16)     →  (102)
(17)     →  (103)
(18)     →  (104)
(19)     →  (105)
```

Triple table (unordered, referenced by pointer):

Addr	op	arg1	arg2
(100)	^	e	f
(101)	*	b	c
(102)	/	(101)	(100)
(103)	+	a	(102)
(104)	*	b	a
(105)	+	(103)	(104)

(iv) Comparison of Representations

Aspect	Quadruples	Triples	Indirect Triples
(a) Memory usage	Highest — 4 fields per record, plus a name/temp for every result	Lower — 3 fields per record; results referenced by position, no extra temp names needed	Slightly more than triples (extra pointer/statement list), but still far less than quadruples
(b) Ease of optimization	Easiest — a statement can be moved/deleted freely since results are named explicitly and not position-dependent	Hardest — moving or deleting a triple changes the index of every triple after it, breaking references	Easy — statements are reordered by editing the pointer list only; the triple table itself never changes
(c) Indirect triples vs. triples	—	—	Combines the compactness of triples with the flexibility of quadruples: code motion, dead-code elimination and common-subexpression elimination only require reordering pointers, not renumbering every triple

Q4. Three Address Code for Boolean Expression: (a > b) and (c < d) and (e < f)

Using the grammar:

```
B → B1 || B2   |   B1 && B2   |   ! B1   |   E1 relop E2   |   true   |   false
```

Method 1 — Numeric-valued (non short-circuit) three address code

```
t1 = a > b
t2 = c < d
t3 = t1 && t2
t4 = e < f
t5 = t3 && t4      // t5 = 1 if the whole expression is true, else 0
```

Method 2 — Short-circuit (jumping) code

```
100:  if a > b goto 102
101:  goto Lfalse
102:  if c < d goto 104
103:  goto Lfalse
104:  if e < f goto Ltrue
105:  goto Lfalse
Ltrue:  ...     // expression evaluates to true
Lfalse: ...     // expression evaluates to false
```

Short-circuit code avoids evaluating later conjuncts once one operand of && is false, and is the form typically generated by a compiler for use in control statements (if/while).

Q5. Backpatching — if (a > b) { x = y + z } else { x = y - z }

Concept

Backpatching generates jump instructions (if-goto / goto) with the target address left blank, and records their instruction numbers in true-lists / false-lists / next-lists. Once the label of the target code is known, the function backpatch(list, target) fills in that address into every instruction in the list. This avoids a separate pass to fix up jump targets.

(a) Three Address Code generated

```
100:  if a > b goto 102     // B.truelist  = {100}
101:  goto 104              // B.falselist = {101}
102:  x = y + z             // start of S1 (the 'then' block)
103:  goto 105              // unconditional jump past 'else' — added to S.nextlist
104:  x = y - z             // start of S2 (the 'else' block)
105:  ...                   // next statement after the if-else
```

(b) Backpatching process

- Generate B's code first: 'if a>b goto _' (100) and 'goto _' (101) — targets unknown. B.truelist = {100}, B.falselist = {101}.
- Let M1 mark the instruction that begins S1 (label 102, the start of x = y + z). Call backpatch(B.truelist, M1) → instruction 100 is filled in as 'if a > b goto 102'.
- Generate S1's code (102: x = y + z), then emit an unconditional jump (103: goto _) whose target is not yet known; add instruction 103 to S.nextlist so it can later be patched to skip over the else-part.
- Let M2 mark the instruction that begins S2 (label 104, the start of x = y - z). Call backpatch(B.falselist, M2) → instruction 101 is filled in as 'goto 104'.
- Generate S2's code (104: x = y - z).

- After the whole if-else is generated, the address of the next statement (105) becomes known. Call backpatch(S.nextlist, 105) → instruction 103 is filled in as 'goto 105'.

Final labels: L1 = 102 (then-branch), L2 = 104 (else-branch), L3 = 105 (statement following the if-else). This two-pass-free technique lets the compiler emit correct, fully-resolved jump targets in a single pass over the syntax tree.